3. Validate attributes through a block.
4. Change add_checked_attribute to a *Class Macro (117)* named attr_checked that's available to all classes.
5. Write a module adding attr_checked to selected classes through a hook.

"Aren't we supposed to work as a pair?" you ask. "I don't even understand these steps."

"Don't worry," Bill says. "You really only need to learn two things before you start developing: one is a method named eval, and the other is the concept of a Hook Method." He vows to tell you everything you need to know about eval, because eval is necessary for the first development step. You will deal with Hook Methods later.

Kernel#eval

Where you learn that, when it comes right down to it, code is just text.

You already learned about instance_eval and class_eval (in *instance_eval()*, on page 85, and *class_eval()*, on page 107, respectively). Now you can get acquainted with the third member of the **eval* family—a *Kernel Method (32)* that's simply named eval. Kernel#eval is the most straightforward of the three **eval* methods. Instead of a block, it takes a string that contains Ruby code—a *String of Code* for short. Kernel#eval executes the code in the string and returns the result:

Spell: String of Code

ctwc/simple_eval.rb

```
array = [10, 20]
element = 30
eval("array << element")  # => [10, 20, 30]
```

Executing a literal string of Ruby code is a pretty pointless exercise, but the power of eval becomes apparent when you compute your Strings of Code on the fly. Here's an example.

The REST Client Example

REST Client (installed with gem install rest-client) is a simple HTTP client library. It includes an interpreter where you can issue regular Ruby commands together with HTTP methods such as get:

```
⇛ restclient http://www.twitter.com

> html_first_chars = get("/")[0..14]
=> "<!DOCTYPE html>"
```